CN Project Roadmap

Performance Analysis of TCP vs UDP Under Emulated Network Conditions

Course: Computer Networks | **Team size:** 1 | **Platform:** Linux (Kubuntu) | **Timeline:** 14 days

---

Phase 0: Pre-requisites — verify before you build anything

These aren't optional setup steps, they're risk-elimination steps. The single biggest way this project goes wrong is discovering on Day 5 that your environment can't actually do what the plan assumes.

0.1 — Confirm your Linux environment can do network namespaces + netem

Run this sanity check **first**, before writing any Python:

```
ip netns list                          # should run without error
sudo ip netns add testns && sudo ip netns del testns   # create/delete works
tc qdisc show                          # tc is present
lsmod | grep sch_netem                 # netem module loaded (or built-in)
```

If you're on bare-metal Kubuntu this will almost certainly work. If you're on WSL or a restricted VM, namespaces or netem may be unavailable or behave differently — find this out now, not mid-project. There's a fallback (loopback + netem instead of namespaces), but it changes your RTT/loss math (delay doubles, loss hits ACKs too), so know which path you're on from day one.

0.2 — Tooling checklist

```
python3 --version          # 3.x
which tshark wireshark     # packet capture
which ethtool              # to disable offloads
tc -V                      # iproute2 present
python3 -c "import matplotlib, pandas, numpy"   # or pip install --break-system-packages
```

Add yourself to the `wireshark` group or plan to run `tshark` via `sudo` — sort out permissions now, not during a timed capture.

0.3 — Repository scaffold

Create this structure before writing code — it keeps you organized and makes the final zip/submission trivial:

```
project/
├── client/        tcp_client.py udp_client.py
├── server/        tcp_server.py udp_server.py
├── setup/         topology.sh netem.sh reset.sh
├── experiments/   run_experiment.py
├── results/       raw_results.csv processed_results.csv
├── analysis/      graphs.py
├── captures/      *.pcapng
└── README.md
```

Init git now. Commit after every working milestone — this is also your proof-of-work trail for academic integrity if ever questioned.

0.4 — Lock the design decisions that are already made for you

Don't re-litigate these — they're settled:

- netem lives on the **router namespace**, not the endpoints (avoids the TSQ issue).
- Delay is applied on **both** router-facing interfaces (50ms each way ≈ 100ms RTT).
- Loss is applied **only** in the client→server direction.
- Offloads (`tso` , `gso` , `gro`) get turned off on every veth — otherwise captures lie to you.

Phase 1 (Days 1–2): Topology + bare transfers

Goal: prove the network itself works before any measurement logic touches it.

1. Write `setup/topology.sh` — namespaces, veth pairs, addressing, forwarding — but make it idempotent (check-and-create, not blind create) so you can re-run it without errors.
2. Write `setup/reset.sh` to tear down cleanly — you'll run this constantly.
3. Verify topology with plain `ping` between client and server namespaces before touching netem.
4. Write `setup/netem.sh` as a function/script that takes delay and loss as arguments and applies them to the router interfaces.
5. **Verify netem itself** with ping: apply 100ms RTT, confirm ping shows ~100ms; apply 1% loss, confirm ping drop rate is roughly 1%. Do this before trusting any later TCP/UDP number — this is your calibration step.
6. Write the *simplest possible* TCP client/server (connect, send N bytes, close) and UDP client/server (send N bytes, no sequencing yet) and get one successful transfer across the namespace topology.

Checkpoint before moving on: topology up → apply netem → ping confirms it → one TCP transfer works → one UDP transfer works → reset, all reliably. If any of these aren't rock solid, don't proceed.

Phase 2 (Days 3–4): Real instrumentation

Goal: turn the toy clients/servers into measurement instruments.

TCP side:

- Server timestamps `t_first_byte` and `t_last_byte` , counts bytes, sends a 1-byte application ACK at the end.
- Client also times until it receives that application ACK (client-side transfer time).

UDP side (the fiddly part — budget real time for it):

- Fixed-size datagrams (~1200 bytes payload) with a 32-bit sequence number header.
- **Paced sending** — a fixed rate *below* your bottleneck bandwidth. Without pacing, loss you observe could

be self-inflicted (send buffer overflow) rather than netem's configured loss, which would wreck your whole loss-sweep story.

- Server: enlarge `SO_RCVBUF`, record every sequence number received, compute missing/duplicate/out-of-order counts.
- End-of-transfer marker sent multiple times (a single marker packet can itself be lost).
- Server-side idle timeout as a safety net.

Both: CSV row output per run (protocol, condition, trial, transfer_time, goodput, retransmissions/loss, etc.) — design this schema now, since Phase 3's automation depends on it.

Checkpoint: a single manual run of TCP and a single manual run of UDP, each producing a correct CSV row, under at least one non-zero delay/loss condition.

Phase 3 (Days 5–6): Automation + pilot

Goal: one script that can run an entire sweep unattended.

1. `experiments/run_experiment.py`: loop over `(condition × protocol × trial)` — apply netem → start `tshark` capture → run server, then client → stop capture, save `.pcapng` with a descriptive name (e.g. `tcp_loss1pct_trial3.pcapng`) → append CSV row → reset netem.
2. Integrate `tshark` retransmission counting using `tcp.analysis.retransmission || tcp.analysis.fast_retransmission`, and cross-check against the kernel's own counters (`ss -ti` or `nstat`) inside the client namespace — document any discrepancy, don't just pick whichever number looks better.
3. **Run a pilot** (small scale, 1–2 trials) across your planned conditions. Use this to freeze your final parameters: payload size, pacing rate, bottleneck bandwidth. At 1% loss, ~700 datagrams gives only ~7 expected losses (noisy); ~8,000 datagrams gives ~80 (stable) — this is why 10MB payloads are recommended for the loss sweep specifically. Confirm this works for your machine's actual transfer speed before locking it in.

Checkpoint: the driver runs start-to-finish with zero manual intervention and produces a clean CSV plus a folder of named captures.

Phase 4 (Days 7–9): Full experiment sweeps

Run E1–E3 for real, 5 trials per point (more if variance looks high):

Set	Sweep	Fixed
E1 Baseline	—	0 delay, 0% loss
E2 Delay	RTT: 0/50/100/200ms	0% loss
E3 Loss	0/0.5/1/2/5%	small fixed delay (~20ms RTT)

Let TCP and UDP use identical machine, payload, and measurement method — the comparison is only valid if everything else is held constant. Sanity check as you go: does observed UDP loss track the configured loss

if everything else is held constant. Sanity-check as you go: does observed UDP loss track the configured loss percentage? If it doesn't, stop and debug pacing/buffers before burning more trials on a broken setup.

If time allows and everything else is solid, E4 (payload size sweep) and the reliable-UDP stretch goal are explicitly optional — cut these first if time runs short.

Phase 5 (Day 10): Analysis and graphs

Build `analysis/graphs.py` to produce these five graphs:

1. Baseline goodput, TCP vs UDP
2. Transfer time vs added RTT (both protocols)
3. Goodput vs loss (both protocols, with error bars; optionally overlay the Mathis curve: `BW ≈ (MSS/RTT) × (1.22/√p)`)
4. TCP retransmissions vs loss
5. UDP observed loss vs configured loss (should track the diagonal)

Where results disagree with theory (e.g., CUBIC vs the Reno-based Mathis model), that's a finding to explain, not an error to hide.

Phase 6 (Days 11–12): Report

Write the 10–12 page report following this structure:

Abstract → Introduction → Problem Statement → Background → Related Work → Methodology → Expected Behaviour → Results → Discussion → Limitations.

The Discussion section should, for each experiment, answer: what was measured, why (which mechanism), was it consistent across trials, does it answer the guiding question, what limits the conclusion — don't just restate table values.

Phase 7 (Day 13): Slides + demo

10–12 slides, live demo plus a **recorded backup** (non-negotiable for a live networking demo — namespaces and netem can fail in front of an audience for reasons that have nothing to do with your code).

Phase 8 (Day 14): Buffer

Catch-up for anything that slipped, or the reliable-UDP stretch goal if everything else is already solid.

Deliverables checklist

- ⬛ Source code (client/, server/, setup/, experiments/, analysis/)
- ⬛ Raw and processed CSV results

- ⬛ Packet captures (`.pcapng`)
- ⬛ Five required graphs
- ⬛ Report (10–12 pages)
- ⬛ Slide deck (10–12 slides)
- ⬛ Live demo + recorded backup
- ⬛ README (requirements, commands, parameters, how to reproduce)